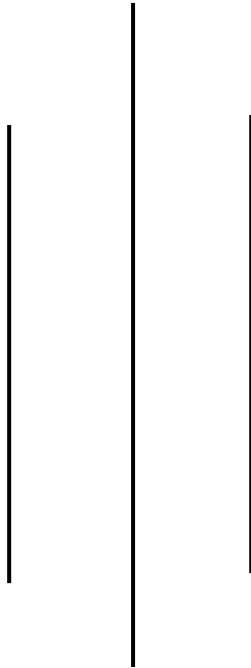


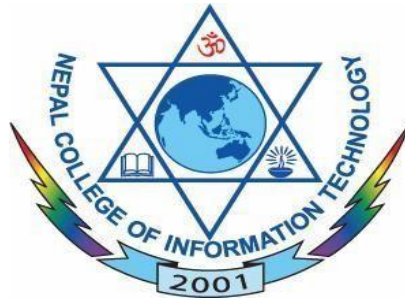
NEPAL COLLEGE OF INFORMATION TECHNOLOGY

Balkumari, Lalitpur

Affiliated to Pokhara University



LAB REPORT FOR EMBEDDED SYSTEMS



LAB REPORT 3: UNIVERSAL GATES AND SPECIAL GATES

Submitted by:

Name: Arpan Adhikari

Roll No.: 231309

BE-CE (5th SEM)

Submitted to:

Er. Simanta Kasaju

S. N	LAB TOPIC	DATE	REMARKS
1.	VHDL Code (OR & AND Gate)	20 th Nov	
2.	VHDL Code (OR & NOT Gate) using testbench	23 rd Nov	
3.	Universal Gates and Special Gates	30 th Nov	

LAB 3: UNIVERSAL GATES AND SPECIAL GATES

OBJECTIVE:

1. To study and understand the working principles of universal gates (NAND and NOR).
2. To verify that NAND and NOR gates can perform the operations of basic logic gates.
3. To write and simulate VHDL code for universal gates.
4. To study the working of special gates using VHDL.
5. To observe the simulation waveforms and verify truth table outputs.

THEORY:

Universal Gates (NAND & NOR)

Universal gates are logic gates that can be used to construct any Boolean function or digital logic circuit. The two universal gates are **NAND** and **NOR**.

NAND Gate

The NAND gate is an AND gate followed by a NOT gate.

- The output is **LOW only when both inputs are HIGH**.

A B NAND

0	0	1
0	1	1
1	0	1
1	1	0

A NAND gate can be used to construct all basic gates (NOT, AND, OR), making it universal.

NOR Gate

The NOR gate is an OR gate followed by a NOT gate.

- The output is **HIGH only when both inputs are LOW**.

A B NOR

0	0	1
0	1	0
1	0	0
1	1	0

A NOR gate can also produce all basic gates, and therefore it is also universal.

Special Gates

Special gates are advanced logic gates used for specific logical operations. Common special gates include:

- **XOR (Exclusive OR)** – Outputs 1 if inputs are different.

- **XNOR (Exclusive NOR)** – Outputs 1 if inputs are the same.
 - **Buffer** – Passes input directly to output.
- These gates are widely used in arithmetic circuits, comparators, parity checkers, and digital

VHDL CODE FOR UNIVERSAL GATES:

```
library IEEE;
use IEEE.Std_Logic_1164.all;

Entity Gate is
    port(
        A, B : in Std_Logic;
        nandop, norop : out Std_Logic
    );
end Gate;
-- nandop and norop is output for NAND and NOR respectively

architecture archGate of Gate is
begin
    process(A,B)
    begin
        if ( A = '1' and B = '1') then
            nandop <= '0';
        else
            nandop <= '1';
        end if;

        if ( A = '0' and B = '0') then
            norop <= '1';
        else
            norop <= '0';
        end if;
    end process;
end archGate;
```

TEST BENCH FOR UNIVERSAL GATES:

```
library IEEE;
use IEEE.Std_logic_1164.all;

Entity test_gates is
end test_gates;

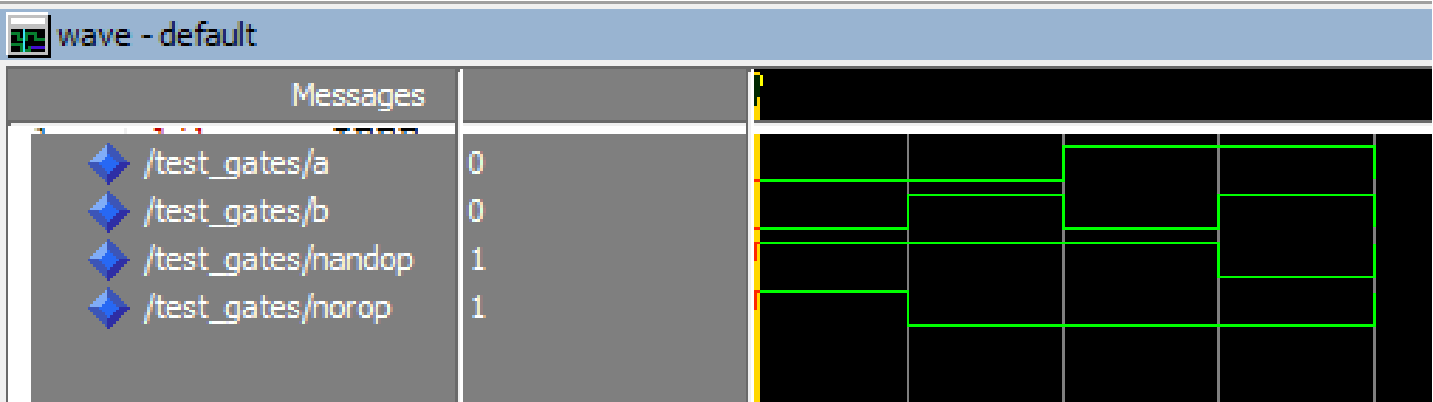
Architecture arpgatebehavioral of test_gates is
component arpgate
  port(
    A, B : in Std_Logic;
    nandop, norop : out Std_Logic
  );
end component;

Signal A,B : Std_logic := '0';
Signal nandop, norop : Std_Logic;

begin
UUT: arpgate port map
(
  A => A,
  B => B,
  nandop => nandop,
  norop => norop
);

stim_proc: process
begin
  A<='0'; B<='0';
  wait for 100 ns;
  A<='0'; B<='1';
  wait for 100 ns;
  A<='1'; B<='0';
  wait for 100 ns;
  A<='1'; B<='1';
  wait for 100 ns;
end process;
end arpgatebehavioral;
```

Waveform for Universal Gates:



VHDL CODE FOR SPECIAL GATES:

```
library IEEE;
use IEEE.Std_Logic_1164.all;

Entity arpagate is
  port(
    A, B : in Std_Logic;
    xorop, xnorop : out Std_Logic
  );
end arpagate;
-- xorop and xnorop is output for XOR and XNOR respectively

architecture archGate of arpagate is
begin
  process(A,B)
  begin
    if ( A = '1' and B = '1') or ( A = '0' and B = '0') then
      xorop <= '0';
      xnorop <= '1';
    else
      xorop <= '1';
      xnorop <= '0';
    end if;
  end process;
end archGate;
```

TEST BENCH FOR SPECIAL GATES:

```
library IEEE;
use IEEE.Std_logic_1164.all;

Entity test_gates is
end test_gates;

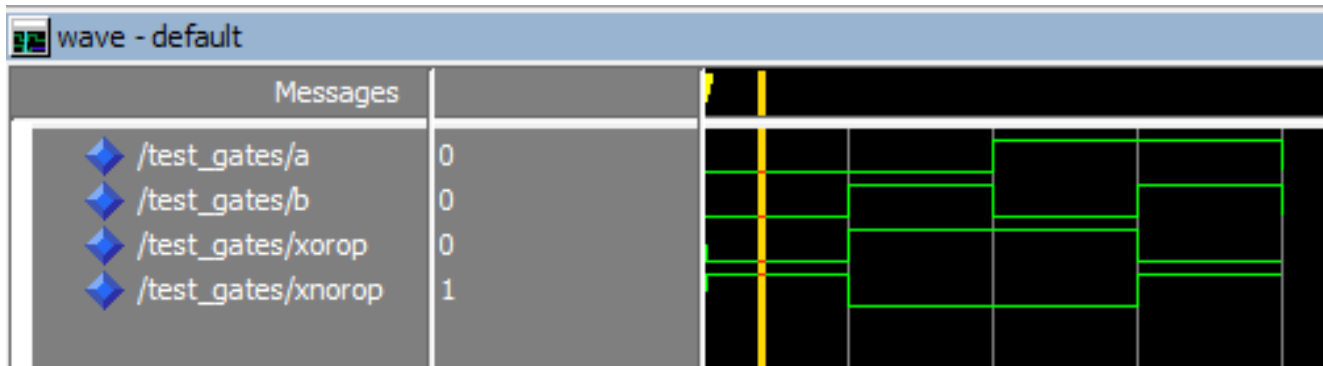
Architecture arpagatebehavioral of test_gates is
  component gates
    port(
      A, B : in Std_Logic;
      nandop, norop : out Std_Logic
    );
  end component;

  Signal A,B : Std_logic := '0';
  Signal nandop, norop : Std_Logic;

begin
  UUT: gates port map
  (
    A => A,
    B => B,
    nandop => nandop,
    norop => norop
  );

  stim_proc: process
  begin
    A<='0'; B<='0';
    wait for 100 ps;
    A<='0'; B<='1';
    wait for 100 ps;
    A<='1'; B<='0';
    wait for 100 ps;
    A<='1'; B<='1';
    wait for 100 ps;
  end process;
end arpagatebehavioral;
```


Waveform for Special Gates:



RESULT:

- VHDL codes for NAND, NOR, and special gates were successfully written and simulated.
- The output waveforms obtained from simulation matched the theoretical truth tables.
- The test benches correctly generated all required input combinations.
- Both universal gates and special gates worked as expected during simulation.

CONCLUSION:

The experiment verified that **NAND** and **NOR** gates are truly **universal gates**, as they can implement any logical function.

The VHDL simulations confirmed the theoretical truth tables for NAND, NOR, XOR, XNOR, and other special gates.

This practical demonstrated how VHDL can be used effectively to design, simulate, and test digital logic circuits before hardware implementation.